

Notebook Build Audit & Execution System

Construction, Audit, and Intelligent LLM Evaluation

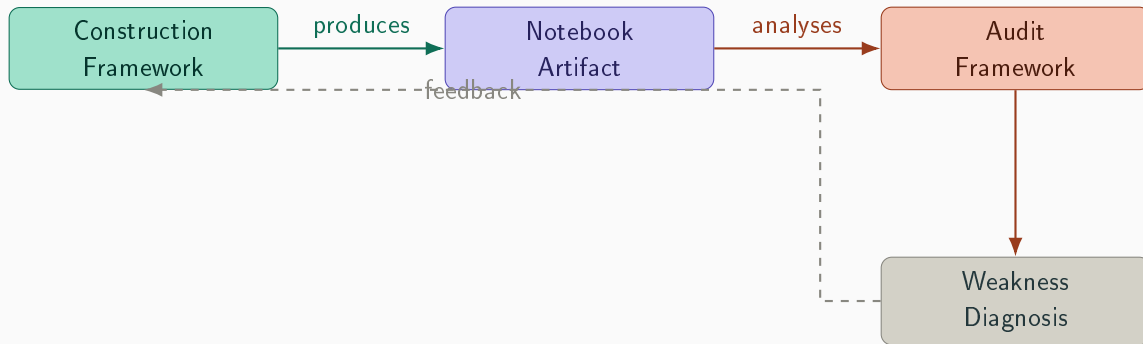
The Challenge

Computational notebooks (Jupyter) are the dominant medium for DS/ML — but they introduce systemic risks:

- **Untrusted execution** — notebooks from collaborators, open-source, or AI assistants without deterministic safety guarantees
- **Lack of deterministic builds** — ad hoc environment pinning, missing dependencies, absent seed controls → different results across machines
- **No structured audit protocol** — no systematic methodology for reviewing notebooks before production, publication, or merge

“Existing code review practices are insufficient for the unique failure modes of notebook-based ML pipelines.”

Two Complementary Frameworks



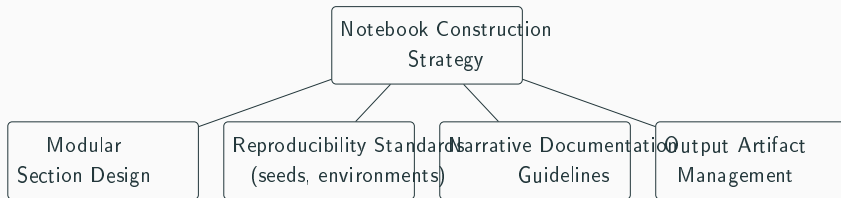
Construction Framework

Forward-looking — guides the author to produce a reproducible, well-structured notebook from the ground up.

Audit Framework

Backward-looking — systematically assesses quality, correctness, and risk of an existing notebook

Construction Framework — Three Phases



Phase 1 — Scaffold

- Single responsibility
- 8 canonical section headers
- Pin environment upfront
- Global reproducibility controls

Phase 2 — Write

- One section at a time
- Markdown intent per code cell
- Explicit variable passing
- Three-block refactor threshold

Phase 3 — Validate

- Restart kernel per section
- Cell idempotency check
- Linear execution order
- Versioned artifact routing

LLM-as-a-Judge — Two Evaluation Layers

Why LLM-based evaluation? Notebook auditing combines **objective** and **subjective** tasks.

Layer 1 — Deterministic

- Dependency validation
- Execution reproducibility
- Train/test split verification
- Artifact existence

Binary, reproducible — programmatic checks

Layer 2 — LLM Judge

- Documentation quality
- Conceptual coherence
- Methodological soundness
- Deployment readiness

Semantic reasoning — structured evaluation prompts

Key insight: Decompose evaluation into explicit criteria (G-Eval). Known biases: position, verbosity, self-enhancement.

ROBIN-HOOD — The Algorithm

Paper: Saha, Wagde & Kveton (2026) — *LLM-as-Judge on a Budget*

Problem: K items, budget B queries — how to allocate to minimise error?

Key insight: High-variance items need more queries; stable items need fewer.

Algorithm (adaptive):

- 1 Evaluate every item once
- 2 Compute μ_i and variance σ_i^2 per item
- 3 Next query $\rightarrow \arg \max_i \sigma_i^2 / n_i$
- 4 Repeat until budget exhausted

Theoretical guarantee:

$$\tilde{O} \left(\sqrt{\frac{\sum_{i=1}^K \sigma_i^2}{B}} \right)$$

- Same accuracy at half the budget
- Tested on *SFF* & *HelpSteer2*

“Leverages multi-armed bandit theory to dynamically concentrate queries where uncertainty is highest.”

ROBIN-HOOD — Application to 6-Pass Audit

Different passes have different variance profiles:

Pass	Type	Allocation
1 — Structural Overview	Easy	Low
2 — Reproducibility Check	Easy	Low
3 — Data Integrity Review	Medium	Medium
4 — ML Correctness Audit	Hard	High
5 — Code Quality Review	Medium	Medium
6 — Deployment Readiness	Hard	High

Impact: Dynamically allocates to Passes 4 & 6 instead of uniform. Same precision,
~50% fewer queries.

“Unevaluated variance wastes budget. Measure it — then allocate.”

The Metonym Game — A Discovery

Paper: David Nordfors (2026) — *The Metonym Game*

What is it? A competitive word game where LLMs generate and evaluate analogies.
No fixed test set — contamination-resistant by design.

SVD on the rating matrix

- K models act as both generators and judges
- Build $K \times K$ rating matrix
- Singular Value Decomposition reveals latent skills

Two key findings

1. Factual correlation with GPQA
Diamond: $r = 0.92$ (no ground truth needed)
2. **Generating $\neq$ Evaluating** — dissociated skills

Dissociation finding

The best generators are middling judges. The sharpest judge is a mid-pack generator.
Never assume the same model excels at both tasks.

Multi-Judge Budget Allocation

Paper 2: *Instance-Optimal Estimation with Multiple LLM Judges on a Budget*

Extends ROBIN-HOOD: multiple judges available, each with different cost & accuracy.

Decision space doubles:

- Which items to evaluate
- Which judge for each query

Rule of thumb:

- Cheap judges for easy items
- Expensive judges for hard items
- Repeat only when uncertainty is high

Theoretical result:

- Instance-optimal: no strategy achieves lower error with same budget
- One right judge per item can beat combining multiple on every item

Practical: GPT-5 for hard passes, Gemini-2.5 for routine, small local model for trivial ones.

Panel Architecture — Implementing the Findings

Both papers conclude: **a panel of diverse judges, not a single LLM.**

ROBIN-HOOD says	Metonym Game says
Allocate budget by variance	Use diverse models
Concentrate on hard items	Gen $\neq$ Evaluators
Same accuracy, half cost	Panels beat individuals

Scheduler (ROBIN-HOOD)

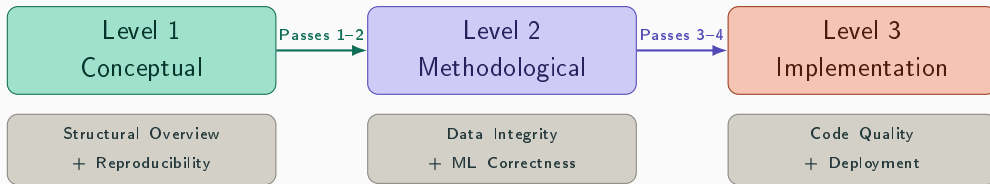
- Tracks variance per audit pass
- Decides **how many** queries

Panel (Metonym-inspired)

- GPT-5, Claude, Gemini, Llama, Mistral
- Decides **which model** per query
- SVD tracks competence over time

ROBIN-HOOD decides quantity; Panel decides quality.

Prompt Crafting — Three-Level Structure



Gate 1 → 2

Notebook fails end-to-end?
Flag or proceed.

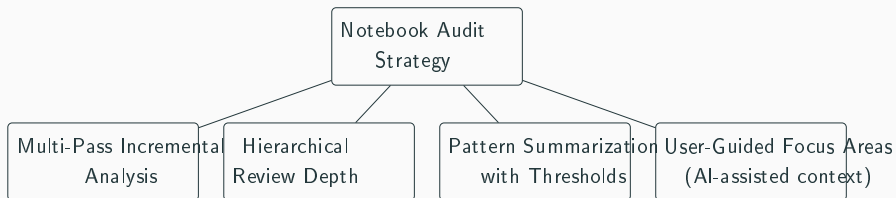
Gate 2 → 3

Methodological flaws (leakage,
invalid CV)? Halt with caveats.

After Level 3

Final report: risk scores across
6 passes, recommendations.

Six-Pass Audit Protocol



Level 1 — Conceptual

- ❶ **Pass 1** — Structural Overview
- ❷ **Pass 2** — Reproducibility Check

Deliverables: Section map, red flags

Level 2 — Methodological

- ❸ **Pass 3** — Data Integrity Review
- ❹ **Pass 4** — ML Correctness Audit

Deliverables: Pipeline report, checklist

Level 3 — Implementation

- ❺ **Pass 5** — Code Quality Review
- ❻ **Pass 6** — Deployment Readiness

Deliverables: Smell report, readiness score

Architecture Overview

Concept

- Construction ↔ Audit lifecycle
- Three-level audit with HITL gates
- Provider-agnostic LLM layer

Physical

- Docker Compose (5 services)
- Ephemeral sandbox per notebook audit
- Local or cloud (Ollama/OpenAI)

Logical

- React + TypeScript Dashboard
- FastAPI REST + SSE Gateway
- **ROBIN-HOOD Scheduler**
- **LLM Panel Proxy** (Strategy Pattern)
- Audit State Machine
- @jupyter-kit Execution Engine
- PostgreSQL + SQLAlchemy

Cost Analysis and ROI

Monthly inference cost — 10,000 item evaluations

Approach	Queries	Monthly cost
Uniform (5 each)	50,000	\$1,500
Uniform (1 each)	10,000	\$300 (poor)
ROBIN-HOOD	~15,000	~\$450
ROBIN-HOOD + Panel	~12,000	~\$350

Period	Without	With ROBIN	Savings
Month 1	\$1,500	\$1,000 + \$500 dev	\$0
Month 2	\$1,500	\$450	\$1,050
Annual	\$18,000	\$11,400	\$6,600

ROI: 37% year 1, ~70% year 2+. At 100K items/month: ~\$10K/month savings.

LLMOps Pipeline

How ROBIN-HOOD + Metanym Game integrate into continuous evaluation:

Monitoring & Observability

- Cost per pass and per judge model
- Variance tracking per audit item
- Judge competence drift detection

Feedback Loops

- ROBIN-HOOD reallocates budget based on drift
- Panel rebalances as competence changes
- HITL triggers when variance exceeds threshold
- Periodic SVD recalibration

CI/CD Integration

- New model → automatic evaluation by panel
- Competence score → deploy / reject
- Score drops → rollback + alert

Key Metrics Dashboard

- Query budget spent per pass
- Variance reduction over time
- Judge panel accuracy matrix
- Cost: ROBIN vs uniform

Papers Comparison

Three papers, one unified strategy

Aspect	ROBIN-HOOD	Multi-Judge	Metonym
Adaptive budget	✓	✓	—
Single judge	✓	✓	—
Multiple judges	—	✓	✓
Different judge costs	—	✓	—
Gen & Eval dissociation	—	—	✓
Optimality guarantee	Partial	✓	—
Panel architecture	—	Implied	✓

Unified takeaway

ROBIN-HOOD → quantity. **Multi-Judge** → model. **Metonym** → diverse panel. Together: max precision, min cost.

Resource Costs

Computational

Item	USD
Server PC	\$1,000
Hosting (3mo)	\$300
Total Q1	\$1,300

Development: 8 weeks, 2 developers

Total: \$5,000 USD

Energetic — Third Pillar Proposal

- LLM inference is energy-intensive (0.01–0.1 kWh per pass)
- Notebook execution often wasteful
- ESG reporting increasingly required

Proposed compromise:

- **Pass 7** (optional) — Energy Impact Assessment
- Backends: RAPL, NVML, powermetrics
- Graceful degradation when unavailable

Timeline & Roadmap

Phase	Summary	Duration
1	Dockerization, monorepo, sandbox spawning, dependency pinning	2 weeks
2	FastAPI REST + SSE, ROBIN-HOOD Scheduler, LLM Panel Proxy, Audit State Machine	3 weeks
3	React dashboard, session management, live progress, risk visualisation	2 weeks
4	E2E testing, panel competence tracking, security review, documentation	1 week

Total: 8 weeks — 2 developers

What do we build first?

Provider Strategy

- Local (Ollama) vs cloud (OpenAI) as default?
- Offline-first or hybrid?

Pass 7 Energy

- Include from Phase 2 as experimental?
- Defer to Phase 4?

Scope

- Full 6-pass from day one?
- Start with Level 1 only?

Audience

- Internal team only, or client-ready?
- Academic publication use case?

Notebook Build Audit & Execution System

Construction

Build right.

3 phases, 8

sections.

Audit

Review any
notebook.

6 passes, 3 levels.

ROBIN- HOOD

Budget allocation.

Half the cost.

Panel

Diverse judges.

Gen $\neq$ Evaluate.

Resource

Track energy.

Optional Pass 7.

Codificar experiencia humana, validar con supervisión sistemática.

-  A. Saha, A. Wagde, and B. Kveton, “LLM-as-Judge on a Budget,” *arXiv:2602.15481*, 2026.
-  D. Nordfors, “The Metonym Game: A Self-Contained, Self-Consistent LLM Peer-Community Benchmark for Structural Intelligence,” *arXiv:2606.21008*, 2026.